



HUS
VNU UNIVERSITY OF SCIENCE

Bài 3: Độ qui và khử độ qui

PGS.TS. Lê Trọng Vĩnh
Khoa Toán – Cơ – Tin học, Trường ĐHKHTN

1. Định nghĩa
2. Khử định nghĩa
3. Giải công thức truy hồi tính độ phức tạp của các thuật toán định nghĩa

1. Định nghĩa

- Khái niệm về giải thuật đệ quy:
 - **Giải thuật đệ quy:** Nếu bài toán T được thực hiện bằng giải thuật của một bài toán T', có dạng giống như T, thì đó là một giải thuật đệ quy.
 - **Ví dụ:** Tính $S(n) = n!$; $S(n) = n * S(n-1)$ vì vậy lời giải của bài toán tính n! được giải thông qua lời giải của bài toán tính (n-1)!
- Đặc trưng của bài toán có thể giải bằng giải thuật đệ quy: Các bài toán phụ thuộc tham số có hai tính chất sau:
 - Ứng với giá trị đặc biệt đó của các tham số thì bài toán có giải trực tiếp (trường hợp suy biến)
 - Trong trường hợp tổng quát ta có thể quy về bài toán tương tự với một bộ giá trị mới của tham số và sau một số hữu hạn bước thì dẫn đến trường hợp suy biến.

1. Định qui

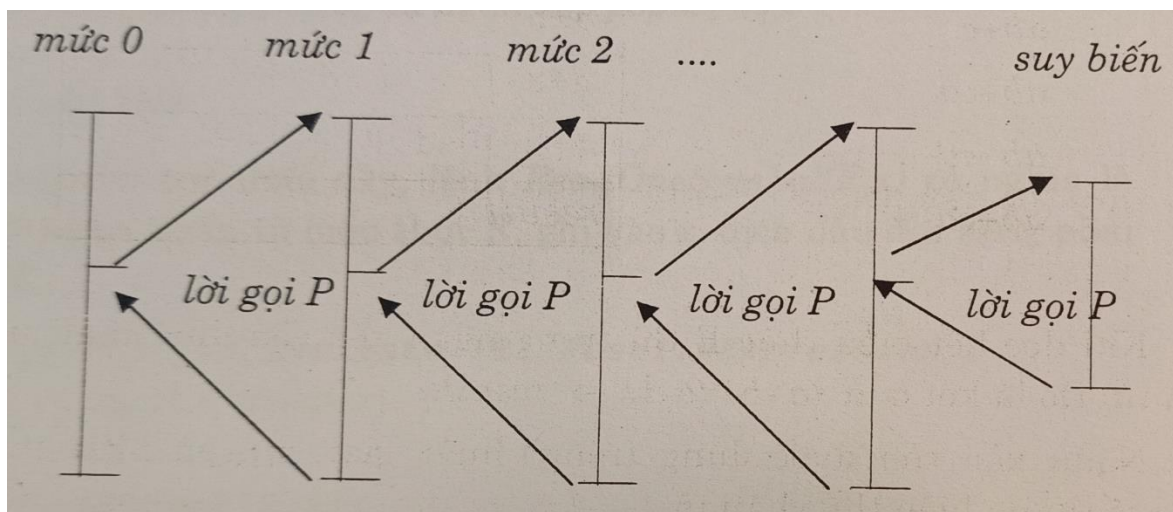
- **Sự thực hiện của các chương trình con lồng nhau:**
 - Mỗi khi gặp chương trình con thì tiến trình thực hiện chương trình chính phải tạm dừng lại. Một tiến trình mới được tạo ra để thực hiện chương trình con.
 - Các thông tin cần thiết để khi kết thúc chương trình con thì quay trở lại tiếp tục thực hiện chương chính từ chỗ đã dừng cần phải được lưu trữ.
 - Máy tính sẽ chứa thông tin này và các biến cục bộ của chương trình con trong khu vực ngăn xếp – Stack.
 - Khi chương trình con kết thúc thì các thông tin trên và các biến cục bộ cũng bị xóa bỏ, vùng nhớ được giải phóng

1. Định qui

- **Sự thực hiện của các chương trình con lồng nhau (tiếp):**
 - Giả sử ta có P_1 chứa P_2 , P_2 chứa $P_3, \dots$
 - Như vậy, vùng nhớ cho P_1 chưa được giải phóng đã phải xếp tiếp vùng dành cho P_2 , rồi tiếp tục đến vùng dành cho $P_3, \dots$
 - Khi chương trình con ở vòng trong cùng, chẳng hạn P_3 , kết thúc thì sẽ giải phóng vùng nhớ dành cho P_3 và quay về thực hiện P_2 . Sau khi P_2 kết thúc sẽ giải phóng vùng nhớ dành cho P_2 và quay về thực hiện P_1
 - Thứ tự giải phóng vùng nhớ là hoàn toàn ngược với thứ tự cấp phát

1. Định qui

- **Sự thực hiện của các chương trình đệ qui:**
 - Thực hiện một chương trình đệ qui tức là thực hiện một dãy các chương trình con lồng nhau cho đến khi gặp trường hợp suy biến



2. Khử đệ qui

- **Khái niệm:**
 - Khử đệ qui tức là bỏ đi các lời gọi đệ qui và thay vào đó sử dụng các cơ chế lặp cùng với sự hỗ trợ của stack.
- **Mục đích:**
 - Chủ động quản lý bộ nhớ.
- **Khử một số dạng đệ qui thường gặp:**
 - Không phải dạng đệ qui nào cũng khử được?

2. Khử đệ qui

- **Khử đệ qui dạng ED[AP]**

```
P(x)=  if E(x) then
        D(x)
      else
        A(x)
        P(f(x))
      endif
```

Trong đó:

- $E(x)$: điều kiện ứng với trường hợp suy biến
- $D(x)$, $A(x)$ là các thành phần không đệ qui
- $f(x)$: thay đổi giá trị của x sau mỗi lần gọi đệ qui

2. Khử đệ qui

- **Khử đệ qui dạng ED[AP] (tiếp)**

Nếu ta thay P bằng chính phần thân của thuật toán, đặt $x:=f(x)$ và triển khai chi tiết ta có:

```
P(x)=    if E(x) then
           D(x)
        else
           A(x)
           x:= f(x)
           if E(x) then
               D(x)
           else
               A(x)
               x:=f(x)
               ....
           endif
        endif
    endif
```

2. Khử đệ qui

- **Khử đệ qui dạng ED[AP] (tiếp)**

- Lưu đồ thuật toán
- Chuyển thành vòng lặp

$P(x) =$ WHILE not $E(x)$ DO

$A(x)$

$x := f(x)$

ENDWHILE

$D(x)$

2. Khử đệ qui

- **Khử đệ qui dạng ED[AP] (tiếp)**

Ví dụ: Phân tích một số nguyên thành các thừa số nguyên tố và in ra theo thứ tự tăng dần:

Phantich(n)=

 d:=2;

 if n=1 then exit

 else

 while n mod d <>0 do d:=d+1 endwhile

 write (d)

 phantich (n div d)

 endif

2. Khử đệ qui

- **Khử đệ qui dạng ED[AP] (tiếp)**

Ví dụ: Phân tích một số nguyên thành các thừa số nguyên tố và in ra theo thứ tự tăng dần:

Phantich(n)=

 d:=2;

 if n=1 then exit

 else

 while n mod d <>0 do d:=d+1 endwhile

 write (d)

 phantich (n div d)

 endif

2. Khử đệ qui

- Khử đệ qui dạng ED[AP] (tiếp)

Ví dụ (tiếp): áp dụng qui tắc khử đệ qui, thuật toán khử đệ qui được viết như sau:

E(x): $n=1$

D(x): exit

A(x): $d:=2$

while $n \bmod d \neq 0$ do $d:=d+1$ endwhile

write(d)

f(x): $n \div d$

2. Khử đệ qui

- **Khử đệ qui dạng ED[AP] (tiếp)**

Ví dụ (tiếp): Ta thấy thuật toán có dạng ED[AP] khi đặt tương ứng:

```
Phantich(n)=
```

```
    d:=2;
```

```
    while n>1 do
```

```
        while n mod d <>0 do d:=d+1 endwhile
```

```
        write (d)
```

```
        n:= n div d
```

```
    endwhile
```

```
    exit
```

2. Khử đệ qui

- Khử đệ qui dạng ED[APB]

```
P(x)=  if E(x) then
        D(x)
      else
        A(x)
        P(f(x))
        B(x)
        P(g(x))
      endif
```

Trong đó:

- E(x): điều kiện ứng với trường hợp suy biến
- D(x), A(x), B(x) là các thành phần không đệ qui
- f(x): thay đổi giá trị của x sau mỗi lần gọi đệ qui

2. Khử đệ qui

- **Khử đệ qui dạng ED[APB] (tiếp)**

Nhận xét:

- Chứng nào $P(f(x))$ chưa kết thúc thì $B(x)$ chưa được thực hiện.
- Để khử đệ qui cho dạng này, chúng ta phải tìm cách cất giữ các giá trị $x, f(x), f(f(x)), \dots$ cho đến khi điều kiện $E(x)$ trở thành đúng.
- Sau đó thực hiện $D(x)$ và lần lượt lấy các giá trị x đã cất giữ (theo chiều ngược lại) để thực hiện $B(x)$

Giải quyết: Sử dụng ngăn xếp S để lưu các giá trị $x, f(x), f(f(x)), \dots$

2. Khử đệ qui

- Khử đệ qui dạng ED[APB] (tiếp)

```
P(x)=  START (S)
        WHILE not E(x) DO
            A(x)
            Push (x, S)
            x:=f(x)
        ENDWHILE
        D(x)
        WHILE not is_empty(S) DO
            pick (S,x)
            B(x)
        ENWHILE
```

2. Khử đệ qui

- **Khử đệ qui dạng ED[APB] (tiếp)**

Ví dụ: Viết ra biểu diễn nhị phân của một số nguyên không âm

Int2Bin=

 if($n > 0$) then

 Int2Bin($n \text{ div } 2$)

 write ($n \text{ mod } 2$)

 end

2. Khử đệ qui

- Khử đệ qui dạng ED[APB] (tiếp)

Ví dụ: Thuật toán đệ qui có dạng ED[APB] khi đặt tương ứng:

$E(x): n=0$

$D(x):$

$A(x):$

$f(x): n \text{ div } 2$

$B(x): \text{write } (n \bmod 2)$

2. Khử đệ qui

- **Khử đệ qui dạng ED[APB] (tiếp)**

Ví dụ: Áp dụng qui tắc khử đệ qui, ta có thuật toán không đệ qui như sau

S: array[1..50] of integer;

st: integer;

IntToBin=

st:=0;

while n>0 do

st = st +1;

S[st]:= n;

n:= n div 2 ;

endwhile

while st <> 0 do

write (S[st] mod 2)

st :=st-1;

endwhile

3. Tính độ phức tạp

- Thời gian thực hiện thuật toán đệ qui được viết dưới dạng công thức truy hồi.
- Công thức truy hồi: Một công thức truy hồi là một đẳng thức hoặc bất đẳng thức mô tả một hàm qua các giá trị với các dữ liệu vào
 - Ví dụ: Độ phức tạp của thuật toán Merge-Sort được mô tả qua công thức sau:

$$T(n) = \begin{cases} 1 & \text{Nếu } n = 1 \\ 2T\left(\frac{n}{2}\right) + n & \text{Ngược lại} \end{cases}$$

3. Tính độ phức tạp

- **Phương pháp thế**
- Công thức 1: Dạng công thức này là các thuật toán đệ qui mà mỗi lần gọi đệ qui kích thước dữ liệu giảm đi 1 và có kiểm tra dữ liệu đầu vào.

$$T(n) = \begin{cases} 1 & \text{Nếu } n = 1 \\ T(n-1) + n & \text{Ngược lại} \end{cases}$$

- Cách giải:

$$\begin{aligned} T(n) &= T(n-1) + n \\ &= T(n-2) + (n-1) + n = T(n-3) + (n-2) + (n-1) + n \\ &\dots \\ &= 1 + 2 + \dots + (n-1) + n \\ &= n \times (n+1) / 2 \end{aligned}$$

Như vậy $T(n) = O(n^2)$

3. Tính độ phức tạp

- **Phương pháp thế**
- Công thức 2: Dạng công thức này là các thuật toán đệ qui mà mỗi lần gọi đệ qui kích thước dữ liệu giảm đi một nửa.

$$T(n) = \begin{cases} 0 & \text{Nếu } n = 1 \\ T(n/2) + 1 & \text{Ngược lại} \end{cases}$$

- Cách giải:
 - Đặt $n = 2^k$ thay vào ta có
 - $T(2^k) = T(2^{k-1}) + 1 = T(2^{k-2}) + 2 = \dots = k$
 - $T(k) = \log_2(k)$ hay $T(n) = \log_2 n$

3. Tính độ phức tạp

- **Phương pháp thế**
- Công thức 3: Dạng công thức này là các thuật toán đệ qui mà mỗi lần gọi đệ qui kích thước dữ liệu giảm đi một nửa nhưng kiểm tra có kiểm tra mỗi phần tử của dữ liệu vào.

$$T(n) = \begin{cases} 0 & \text{Nếu } n = 1 \\ T(n/2) + n & \text{Ngược lại} \end{cases}$$

- Cách giải:

$$\begin{aligned} T(n) &= T(n/2) + n \\ &= T(n/4) + n/2 + n = T(n/8) + n/4 + n/2 + n \\ &\dots \\ &= n + n/2 + n/4 + n/8 + \dots \\ &= 2n \\ T(n) &= O(n) \end{aligned}$$

3. Tính độ phức tạp

- **Phương pháp thế**
- Công thức 4: Dạng công thức này là các thuật toán chia để trị.

$$T(n) = \begin{cases} 0 & \text{Nếu } n = 1 \\ 2T(n/2) + n & \text{Ngược lại} \end{cases}$$

- Cách giải:

Đặt $n=2^k$

$$T(2^k) = 2T(2^{k-1}) + 2^k$$

$$T(2^k)/2^k = T(2^{k-1})/2^{k-1} + 1 = T(2^{k-2})/2^{k-2} + 1 + 1 = \dots = k$$

$$T(2^k) = k2^k \Rightarrow T(k) = k \log_2 k$$

$$\text{hay } T(n) = O(n \log n)$$

3. Tính độ phức tạp

- **Phương pháp chính**

- Phương pháp chính cung cấp lời giải ngay cho trường hợp các công thức đệ qui có dạng

$$T(n) = a T(n/b) + f(n)$$

Với các hằng số $a \geq 1$, $b > 1$ và $f(n)$ là một hàm số tiệm cận dương

- Công thức này mô tả thời gian thực hiện của một thuật toán chia một vấn đề có kích thước n thành a vấn đề con, mỗi vấn đề con có kích thước n/b , và a vấn đề con được giải bằng đệ qui với thời gian thực hiện là $T(n/b)$. Chi phí cho việc chia nhỏ vấn đề và kết hợp các lời giải của vấn đề con là $f(n)$.

- Phương pháp chính phụ thuộc vào định lý sau:

3. Tính độ phức tạp

- **Định lý chính chính**

Cho $a \geq 1$, $b > 1$ là các hằng số. Hàm $T(n)$ được định nghĩa trên tập các số nguyên bởi công thức truy hồi

$$T(n) = a T(n/b) + f(n)$$

Khi đó $T(n)$ có thể có các giới hạn tiệm cận như sau:

1. Nếu $f(n) = O(n^{\log_b a - \epsilon})$ với $\epsilon > 0$, khi đó $T(n) = O(n^{\log_b a})$
2. Nếu $f(n) = O(n^{\log_b a})$, khi đó $T(n) = O(n^{\log_b a} \log n)$
3. Nếu $f(n) = O(n^{\log_b a + \epsilon})$ với $\epsilon > 0$ và nếu $a.f(n/b) \leq c.f(n)$ với hằng số $c < 1$ với mọi số nguyên n đủ lớn thì khi đó $T(n) = O(f(n))$

3. Tính độ phức tạp

- **Ví dụ 1:** Xét công thức

$$T(n) = 9T(n/3) + n$$

Ta có: $a = 9$, $b = 3$ và $f(n) = n$, do đó $n^{\log_b a} = n^{\log_3 9} = n^2$. Vì $f(n) = \log_3 9 - \varepsilon$ với $\varepsilon = 1$, chúng ta có thể áp dụng trường hợp 1 của phương pháp chính và kết luận $T(n) = O(n^2)$.

- **Ví dụ 2:** Xét công thức

$$T(n) = T(2n/3) + 1$$

Ta có: $a = 1$, $b = 3/2$ và $f(n) = 1$, do đó $n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$. chúng ta có thể áp dụng trường hợp 2 của phương pháp chính vì $f(n) = O(n^{\log_b a}) = O(1)$ và kết luận $T(n) = O(\log n)$.

3. Tính độ phức tạp

- **Ví dụ 3:** Xét công thức

$$T(n) = 3T(n/4) + n \log n$$

Ta có: $a = 3$, $b = 4$ và $f(n) = n \log n$, do đó $n^{\log_b a} = n^{\log_4 3} \sim n^{0.793}$. Vì $f(n) = n^{\log_4 3 + \epsilon}$ với $\epsilon \sim 0.2$, chúng ta có thể áp dụng trường hợp 3 của phương pháp chính với điều kiện của $f(n)$: với n đủ lớn $a.f(n/b) = 3(n/4)\log(n/4) \leq 3/4 n \log n = c.f(n)$ với $c = 3/4$. Do vậy $T(n) = O(n \log n)$.